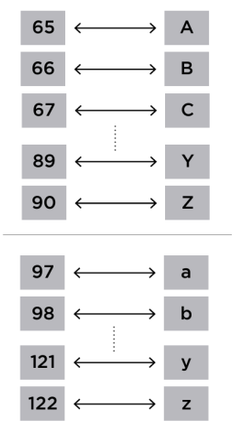
Chapter 3
Formatted Input/Output

CSE 4107 : Structured Programming I
REYNEP

3.0. characters



@credit *cs50*

```c
char c = 'A'
char c = 65;  // same effect as last line

char l = 'b';
char l = 98;  // same effect as last line
```

```c
for (int i = 0; i < n; i++)
{
    char c = str[0];     // if c = '5'
     int k = '0' - c;    //    k = 5;
}
```

ASCII

```
cook@pop-os:~$ ascii -d
   0 NUL    16 DLE    32      48 0    64 @    80 P    96 `   112 p
   1 SOH    17 DC1    33 !    49 1    65 A    81 Q    97 a   113 q
   2 STX    18 DC2    34 "    50 2    66 B    82 R    98 b   114 r
   3 ETX    19 DC3    35 #    51 3    67 C    83 S    99 c   115 s
   4 EOT    20 DC4    36 $    52 4    68 D    84 T   100 d   116 t
   5 ENQ    21 NAK    37 %    53 5    69 E    85 U   101 e   117 u
   6 ACK    22 SYN    38 &    54 6    70 F    86 V   102 f   118 v
   7 BEL    23 ETB    39 '    55 7    71 G    87 W   103 g   119 w
   8 BS     24 CAN    40 (    56 8    72 H    88 X   104 h   120 x
   9 HT     25 EM     41 )    57 9    73 I    89 Y   105 i   121 y
  10 LF     26 SUB    42 *    58 :    74 J    90 Z   106 j   122 z
  11 VT     27 ESC    43 +    59 ;    75 K    91 [   107 k   123 {
  12 FF     28 FS     44 ,    60 <    76 L    92 \   108 l   124 |
  13 CR     29 GS     45 -    61 =    77 M    93 ]   109 m   125 }
  14 SO     30 RS     46 .    62 >    78 N    94 ^   110 n   126 ~
  15 SI     31 US     47 /    63 ?    79 O    95 _   111 o   127 DEL
```

characters

- `char` ➡️ *8-bit*

3.1. The `printf()` function

```c
int i = 10;
int j = 20;
float x = 43.2892f;
float y = 5527.0f;

printf("hello, world!\n");
printf("i = %d, j = %i, x = %f, y = %f\n", i, j, x, y);
```

```
// output
hello, world!
i = 10, j = 20, x = 43.289200, y = 5527.000000
```

"string"

`"hello, world!\n"`

- *This is called a* String
- *Whatever is* `"` *inside* `"` *Double-Quotes* ➡️ String
- `\n` - newline
  - *This is the* ASCII *Character* `10`
  - *Without this, there'd be no 2nd line*

"conversion specification"

- `%d` - *decimal*
- `%i` - *integer*
- `%f` - *float*
  - *These are called* conversion specification
  - *They start with a* `%`
  - *They must stay inside* String
  - *They get "filled in" with values of* `i` , `j` , `x` , `y` *during printing* 🤷‍♀️

"format string"

- *The* `printf()` *function takes a* string *as input*
- *now, because* `printf()` *has built-in functionality to handle* conversion specification *(* `%d` , `%` *stuffs)*
  - *this* string *gets a special name*
  - *this* string *is called* "format string"

- *3 kinds of stuff that can be inside this* "format string"
  - *i.* "ordinary characters" ➡️ *letters/buttons of a keyboard*
    - *simply, English Letters, Digits*
    - *stuff from the ASCII table*
  - *ii.* "escape sequence" ➡️ *stuff like* `\n`
    - `\n` *is also another ASCII character*
  - *iii.* "conversion specifications" ➡️ `%d` , `%` *stuffs*
    - *by using* `%d` , `%` *stuffs, we can insert values @ specified points in the* string *& those values will be printed*

using `%d`

```c
int x = 50;

printf("welcome to cs%d", x);
printf("\n");
```

1.

```c
int x = 10;
int y = 20;
printf("x = %d, y = %d", x, y);
printf("\n");
```

2.

1. *we used 1* `%d` *, so we passed in 1 integer right after the string*
2. *we used 2* `%d` *, so we passed in 2 integer right after the string*
   - *so, like, formally,*
     - *the values of* `x` *,* `y` *are gonna be "inserted into" the format string, "left-to-right" @ the places of conversion specification*

- *There's no limit on "the number of values" that can be printed by 1 single call of* `printf()`

ivan's lines 🟩

- `printf()` *must be supplied with the format string,*
  followed by any values that are to be inserted into the string during printing:

```c
printf(string, expr1, expr2, …);
```

- *The format string may contain both*
  - "ordinary characters" ➡️ *letters/buttons of a keyboard*
  - "conversion specification" ➡️ *which begin with the* `%` *character.*

- *A conversion specification is a placeholder representing a value to be filled in during printing*
  - `%d` *is used for* `int` *values*
  - `%i` *is used for* `int` *values*
  - `%f` *is used for* `float` *values*

- *Then he shows the code @ the start of this page, & typed beside*
  - "ordinary characters" *in a format string are printed as they appear in the string;*
  - "conversion specifications" *are replaced*

Possible Errors

1.

```c
printf("%d %d\n", i);          /*** Too many conversion specifications ***/
```

```
outputs:
   50 1561391600
   50 <some-random-int>
```

- Compilers *are not required to check if the number of conversion specification in a format string matches the number of output items.*

2.

```c
printf("%d\n", i, j);          /*** Too few  conversion specifications ***/
```

```
outputs:
   50
```

- *works,* `j` *is just ignored.*

3.

```c
int k = 50;
float y = 3.14;
printf("%f %d\n", k, y);       /***   wrong  conversion specifications ***/
```

```
outputs:
   0.000000 1610612736
```

- `float` *is stored way differently than* `int`
  - *so, when a really small* `float` *like* `3.14` *is "interpreted as"* `int` *, smth like* `1610612736` *comes out!*
    - *same goes for the other way around.*
- *If the programmer uses an incorrect conversion specification, the program will produce meaningless output, like in this case.*
- Compilers *are not required to check if a conversion specification is appropriate*

3.1.1. "Conversion Specifications" = "Format Specifiers"

- *ivan calls these* "Conversion Specifications"*, by* "Format Specifiers"
  - *iguess, because they specify stuff inside* "Format Strings"
  - *actually, the whole world calls these* `%` *stuffs* "Format Specifiers"

%ক

- Numbers

`%ক`	*Bits*	*Description*	*Range / Notes*	`%llক`	*Bits*	*Range / Notes*
`%i`	*32-bit*	`integer`	*max:* $2^{31} - 1$	`%lli`	*64-bit*	*max:* $2^{63} - 1$
`%u`	*32-bit*	Unsigned `int`	*Positive or zero*	`%llu`	*64-bit*	
`%d`	*32-bit*	Decimal `int` *- Same as* `%i`		`%lld`	*64-bit*	
`%x`	*32-bit*	Hexadecimal *(Lower-case)*		`%llx`	*64-bit*	
`%X`	*32-bit*	Hexadecimal *(Upper-case)*		`%llX`	*64-bit*	
`%o`	*32-bit*	Octal		`%llo`	*64-bit*	
`%f`	*32-bit*	`float`	*max* ≈ 3.4×10^{38}	`%lf`	*64-bit*	*max* ≈ 1.8×10^{308}
`%a`	*32-bit*	Hex `float`	*C99 style*	`%la`	*64-bit*	

  - *we &* `ivan` *will only use* `%i` , `%d` *&* `%f`
  - `64-bit`
    - *integer :* `%lli` , `%lld` , `%llu`
    - *float :* `%lf`
  - `16-bit`
    - `%hi` , `%hd` , `%hu`

- Scientific Notation

`%ক`	*Bits*	*Description*
`%e`	*64-bit*	*Scientific (Lower-case)*
`%E`	*64-bit*	*Scientific (Upper-case)*
`%g`	*64-bit*	*Shortest of* `%f` *or* `%e`

Char, Letters, Strings

`%ক`	*Bits*	*Description*	*Range / Notes*
`%c`	*8-bit*	`char`	*0–127 (ASCII)*
`%s`		`char[] str`	
`%p`	*64-bit*	`pointer`	

- Tricks *- that we won't be using / that might confuse us*

Specifier	*Description*
`%#x`	*Hex with* `0x` *prefix*
`%+d`	*Show sign for positive*
`%05d`	*Zero-padded width 5*

  - `%n` *- Saves The Number of Characters Printed so far by current call of printf() into variable*

```c
int k = 40; float x = 839.21f;

printf("|%05d|\n", k);
printf("|%+d|\n", k);
```

```
// outputs

|00040|
|+40|
```

3.1.2. Gaps

```c
int k = 40;
printf("|%d|%5d|%-5d|%5.3d|-%5.3d|\n", k, k, k, k, k);
```

```
// outputs: using '·' to represent '<space>'
|40|···40|40···|··040|040··|
```

- `%d` - `int` *using minimum amount of space*
- `%5d` - `int` *using minimum of 5 characters.* "Right-Justified" *[1]*
- `%-5d` - `int` *using minimum of 5 characters.* "Left-Justified"
- `%5.3d` - `int` *using minimum of 5 characters.* "Right-Justified"*. At Least 3 Digits. [2]*
- `%-5.3d` - `int` *using minimum of 5 characters.* "Left-Justified"*. At Least 3 Digits.*

- *[1]*
  - *Since variable* `k` *requires only two characters* `40`, three spaces *were added*
- *[2]*
  - *This one is actually confusing.*
  - *So, do some testing if you need.*
  - *Adds extra* "Leading Zeroes" *if needed*

```c
float x = 839.21f;
printf("|%f|\n", x);
printf("|%10f|\n", x);
printf("|%10.3f|%10.3e|%-10g|%-12f\n", x, x, x, x);
```

```
// using '·' to represent '<space>'
|839.210022|
|839.210022|
|···839.210|·8.392e+02|839.21···|839.210022··|
```

- `%f` - `float` *using minimum amount of space, to show the* "actual value"
  - *for our case it shows* `839.210022`
  - *cz binary can't store* `839.21` *exactly. The closest it can store is* `839.210022`
  - *This approximation is called* "floating point error margin"

- `%10f` - `float` *using minimum of 10 characters.*
- `%-12f` - `float` *using minimum of 10 characters.* "Left-Justified"
- `%10.3f` - `float` *using minimum of 10 characters.* "Right-Justified"*. At least 3 digits* "after `.`"
- `%10.3f` - `float` *6chars* `.` *3digits ---------- [6 + 3 + 1 for* `.` *= 10]*
- `%10.3e` - `float` *2chars* `.` *3digits* `e` ± `02`
- `%10.3e` - `float` *2chars* `.` *3digits* `e` *3chars*
- `%15.3e` - `float` *7chars* `.` *3digits* `e` *3chars*
- `%15.7e` - `float` *3chars* `.` *7digits* `e` *3chars*

- `%-10g` - `float` *chooses between* `%f` *&* `%e`*. Omits* "Trailing/Leading Zeroes"
  - *For our example case, chooses* `%f`
  - *Acts exactly like* `%-10.2f`

3.1.3. Gaps - Summaries

- `%5d` v/s `%10f`
  - `%5d` : `int` *using min. 5 chars.* "Right-Justified"
  - `%10f` : `float` *using min. 10 chars.* "Right-Justified". `.` *is an extra char which will always be there.*
  - `%10e` : `exponential` *using min. 10 chars.* "Right-Justified". `e+00` *are 4 extra char.*

- `%5.3d` v/s `%10.3f`
  - `%5.3d` : `int` *using min. 5 chars.* "Right-Justified". *At Least 3 Digits* "using `0`"
  - `%10.3f` : `float` *using min. 10 chars.* "Right-Justified". *At least 3 digits* "after `.`"

3.1.4. `___` $\%-n_1.n_2$क

ex.	generalized
`%i` , `%d` , `%u` , `%f`	%क
`%lli` , `%lld` , `%lf`	% *ll* क
`%5d` , `%15f`	% n_1क
`%5.3d` , `%10.3f`	% n_1 . n_2क
`%-5.3d` , `-%10.3f`	% − n_1 . n_2क

- *ll* : *64-bit stuffs*
- − : "Left-Justified"
- n_1 : *using minimum of* n_1 *chars. (inclusing* `.` *-decimal-point, or* `e+00` *)*
- n_2 : *at least* n_2 *digits*
  - `int` *at least* n_2 *digits* "using `0`"
  - `float` *at least* n_2 *digits* "after `.`"

```c
int k = 40;
float x = 839.21f;

printf("|%d|%5d|%-5d|%5.3d|-%5.3d|\n", k, k, k, k, k);
printf("|%f|\n", x);
printf("|%10f|\n", x);
printf("|%10.3f|%10.3e|%-10g|%-12f\n", x, x, x, x);
```

```
// outputs: using '·' to represent '<space>'


|40|···40|40···|··040|040··|
|839.210022|
|839.210022|
|···839.210|·8.392e+02|839.21···|839.210022··|
```

ivan's lines (edited) 🟩

- *A conversion specification can have the form* $\%n_1.n_2$क *or* $\%-n_1.n_2$क ,
  - क *is a letter.*
  - n_1 *and* n_2 *are integer* constants
  - n_1 *and* n_2 *are optional*
    - *if* n_2 *is omitted then, the* `.` *dot that separates* n_1 *and* n_2 *is also dropped*

- `%10.2f`
  - n_1 *is* `10`
  - n_2 *is* `2`
  - क *is* `f`

- `%10f`
  - n_1 *is* `10`
  - n_2 *is missing (so is the* `.` *)*
  - क *is* `f`

- `%.2f`
  - n_1 *is missing*
  - n_2 *is* `2`
  - क *is* `f`

- $\%n_1.n_2$क $\implies$ "format specifier" $\implies$ $\%m.pX$
  - $n_1 \to m$ $\implies$ "minimum field width"
  - $n_2 \to p$ $\implies$ "precision"
  - क $\to X$ $\implies$ "conversion specification"
    - *I hate ivan's decision of using 'X' capital-x here,*
      because, `%X` *actually has it's own actual meaning "Hexadecimal (Upper-Case)"*

ivan's lines : (unedited) - details

- *Slide 6*
  - *A* "conversion specification" *can have the form* `%m.pX` *or* `%-m.pX`, *where* `m` *and* `p are integer constants and` `X` *is a letter*
  - *Both* `m` *and* `p` *are optional; if* `p` *is omitted, the period that separates* `m` *and p is also dropped*
  - *In the conversion specification* `%10.2f`, `m` *is* `10`, `p` *is* `2`, *and* `X` *is* `f`
  - *In the specification* `%10f`, `m` *is* `10` *and* `p` *(along with the* `.` *period) is missing,*
    - *but in the specification* `%.2f`, `p` *is* `2` *and* `m` *is missing*

- *Slide 7+8 :* $n_1 \rightarrow m$
  - *The* "minimum field width", `m`, *specifies the minimum number of characters to print*
  - *If the value to be printed requires* fewer than m *characters, it is* "left/right-justified" *within the field.*
    - `%5d` - "Right-Justified"
    - `%-5d` - "Left-Justified"
    - `%4d` *displays the number* `123` *as* `•123`.
      - `•` *represents the space character*
  - *If the value to be printed requires* more than m *characters, the field width automatically expands to the necessary size*
    - *don't get confused*
    - *this simply means, in this case,*
      - `%5d` *for* `123456` *the effect is same as* `%d`
  - `%-5d` *Putting a minus sign in front of* m *causes* "left justification"
    - *The specification* `%-4d` *would display* `123` *as* `123•`

- *Slide 9 :* $n_2 \rightarrow p$
  - *The meaning of the* "precision", p, *depends on the choice of* X, *the* "conversion specifier"
  - `%.3d` - *The* `d` *specifier is used to display an integer in decimal form.*
    - p *indicates the* "minimum number of digits" *to display (extra zeros are added to the beginning of the number if necessary).*
    - *If* p *is omitted, it is assumed to be 1*

- *Slide 10 :* $n_2 \rightarrow p$
  - `%e` *- Exponential format*
    - p *indicates how many digits should appear after the decimal point (the default is* `6` *including* `e+00` *???). If* p *is 0, no decimal point is displayed.*
  - `%f` *- Float - Non-Exponential Format (ivan calls 'non-exponential' = 'fixed decimal')*
    - p *has the same meaning as above*
  - `%d` *- Integer*
    - *Minimum number (*p*) of digits to display (append extra zeros at the beginning if needed)*

- *Slide 11*
  - *Show the use of format specifiers for both* `int` *and* `float`
    - *Use* `%d`, `%5d`, `%-5d`, `%5.3d` *for* `int k = 40`
    - *Use* `%10.3f`, `%10.3e` *for* `float y = 839.21`

3.1.5. Escape Sequence

```
printf("Item\tUnit\tPurchase\n\tPrice\tDate\n");
```

```
Item    Unit    Purchase
        Price   Date
```

- *The* `\n` *code that we used in format strings since day 1, is called an* "escape sequence"
- *Control Characters:*
  - *Alert (Bell) :* `\a` *- printing* `\a` *causes an audible beep on most machines. (well, REYNEP never saw it. 🥺)*
  - *NewLine :* `\n` *- printing* `\n` *advances the cursor to the beginning of the next line.*
  - *Horizontal Tab :* `\t` *- printing* `\t` *moves the cursor to the next tab stop*
  - *BackSpace :* `\b` *- printing* `\b` *moves the cursor back one position*

- *Special Characters:*
  - *Backslash :* `\\` - `printf(" ---- \\ ---- ");`
  - *Double Qotes :* `\"` - `printf(" ---- \" ---- ");`

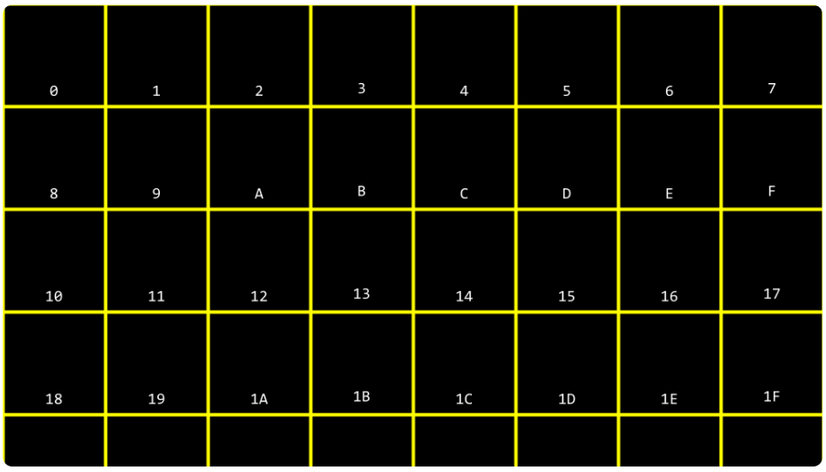
3.1.6. White-Space Characters

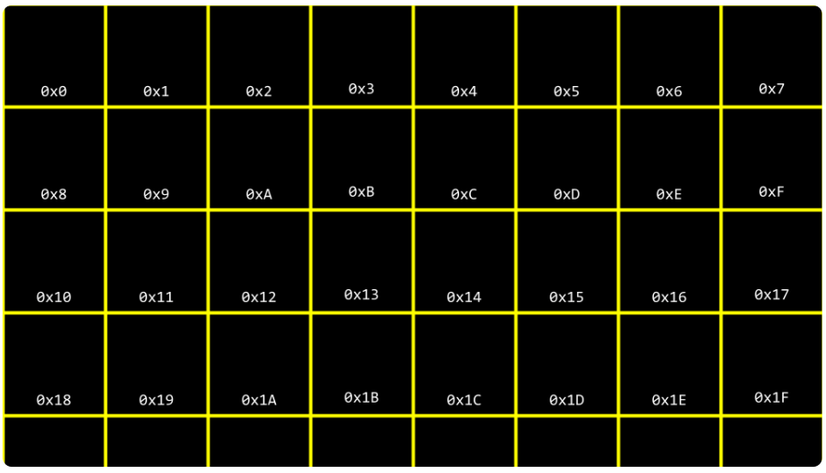
- ` ` - ASCII 32 - **space**
- `\n` - ASCII 10 - **newline** - *Moves cursor to the beginning of the next line*
- `\t` - ASCII 9 - **horizontal tab**
- `\v` - ASCII 11 - **vertical tab**
- `\f` - ASCII 12 - **form feed** - *Advances the output to the next page or form*
- `\r` - ASCII 13 - **carriage return** - *Moves cursor to the beginning of the current line*

3.2.0. Memory

In weeks past, you may recall our "artist rendering" of concurrent blocks of memory.

Applying hexadecimal numbering to each of these blocks of memory, you can visualize these as follows:





You can imagine how there may be confusion regarding:

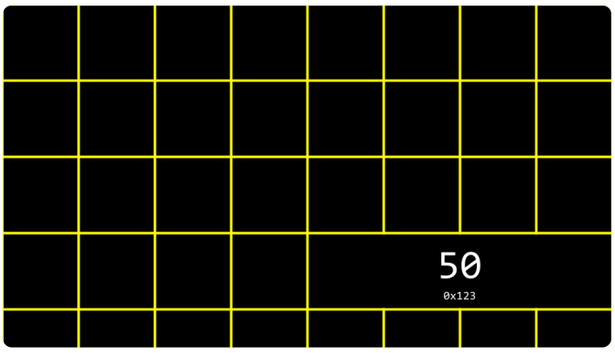
- *whether the memory-block labeled* `10` *in the first picture above*
  - *may represent a location in memory or*
  - *may represent the value 10.*

Accordingly, by convention, all hexadecimal numbers are often represented with the `0x` prefix as follows:

```c
// Prints an integer
#include <stdio.h>

int main(void)
{
    int n = 50;
    printf("%i\n", n);
}
```

Notice how `n` is stored in memory with the value 50.
You can visualize how this program stores this value as follows



Notice how `n` is stored in memory.

- `value(n) = 50`
- `address(n) = 0x123`
  - *There is a way we can Print this Address.*

```c
// Prints an integer's address
#include <stdio.h>

int main(void)
{
    int n = 50;
    printf("%p\n", &n);
}
```

3.2.1. Pointers

The C language has two powerful **operators** that relate to memory:

- `&` - *Provides the address of something stored in memory. [**this page**]*
- `*` - *Instructs the compiler to go to a location in memory. [**next page**]*

```c
// Prints an integer's address
#include <stdio.h>

int main(void)
{
    int n = 50;
    printf("%p\n", &n);
}
```

`n` is stored in memory.

- `value(n) = 50`
- `address(n) = 0x123`
  - *There is a way we can Print this Address.*
  - *won't stay "invariant", if we run it again.*

Things to notice:

- `%p`, *which allows us to view the address of a location in memory.*
- `&n` *can be literally translated as "the address of n."*
- `printf()` *line prints the integer's address.*

Executing this code will print an address of memory beginning with `0x`

> Def ^{n}
>
> > A **"pointer"** is a variable that stores the address of something.

```c
int  n = 50;
int *p = &n;
```

`p` is a pointer that contains the address of an integer `n`
Here, `*` is not an "operator"

- *wait a lil' bit before we get to that* `*` *"operator"*

```c
// Prints an integer's address
#include <stdio.h>

int main(void)
{
    int n = 50;

    printf("%p\n", &n);
}
```

- *previous code*

```c
// Stores and prints an integer's address
#include <stdio.h>

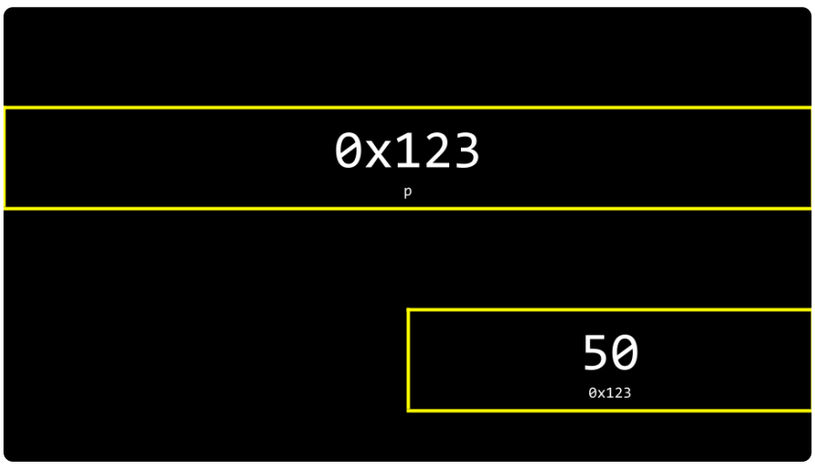
int main(void)
{
    int  n = 50;
    int *p = &n;
    printf("%p\n", p);
}
```

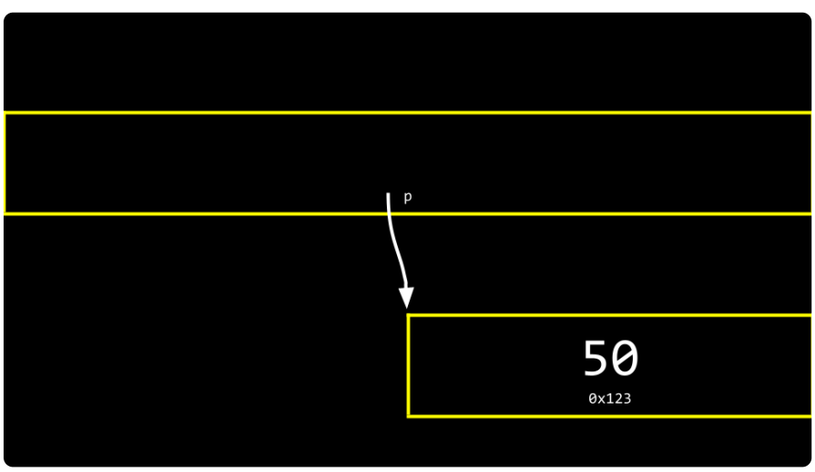
- *modified new code ➡️ has the same effect*

We have simply leveraged our new knowledge of the `&` operator.

Notice that the **pointer** seems rather large

- *it looks double sized in the picture*
- *Indeed, a pointer is usually stored as an 8-byte value.*
  - *8-byte = 64-bit*
  - *Why? ➡️ 🔗LinusTechTips*
- *maybe, by chance* `p` *ends up, here, sort of higher up on my slide*





Modified New Code ⟺ Memory

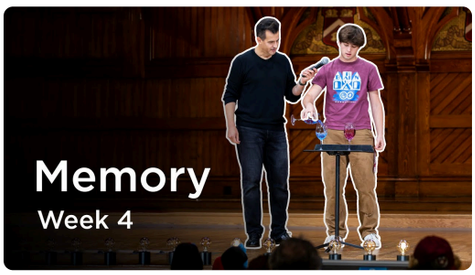
- `int *p` *creates a pointer*
  - *whose job is to store the memory address of an integer.*
- `int *p` *is a variable itself tho*
  - *var* `p` *is storing the address of var* `n`
  - *var* `p` *is storing the address of the* `50`

It is **Arcane**
It's a little weird

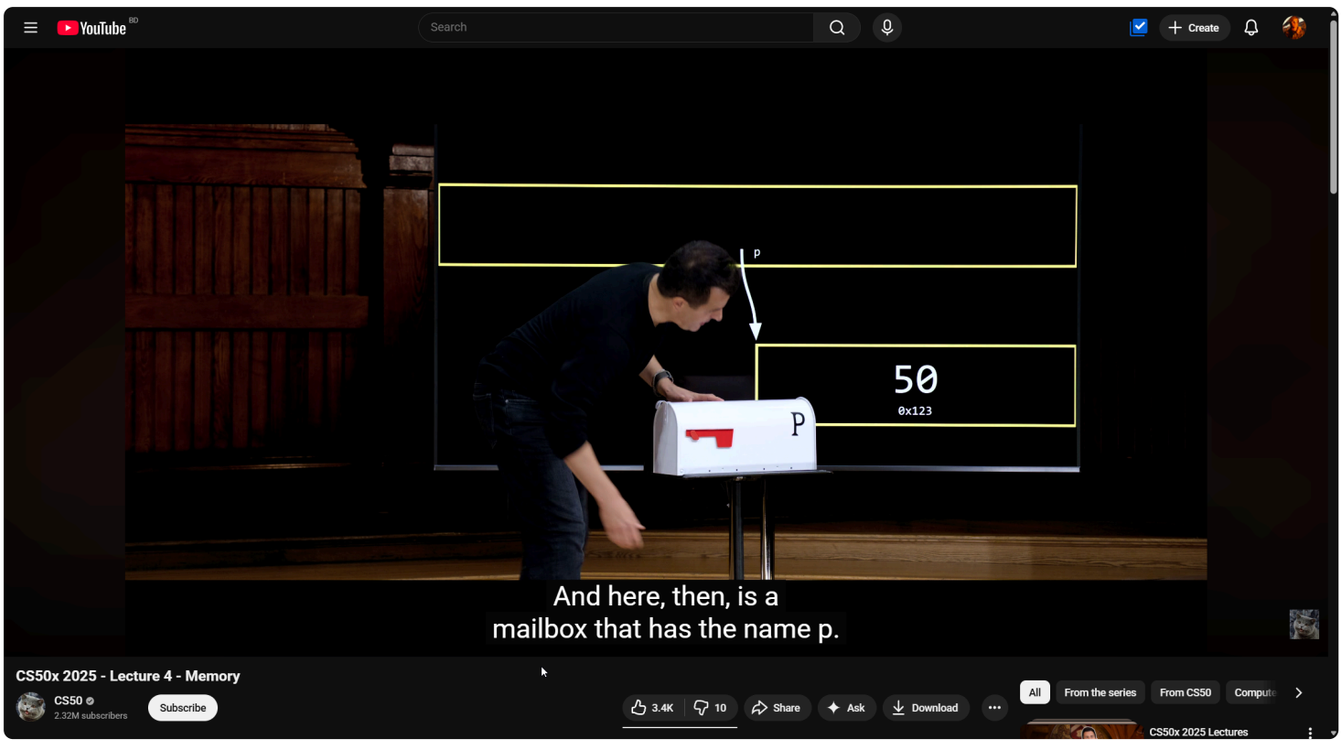
3.2.2. Pointers - `*` Operator - page1

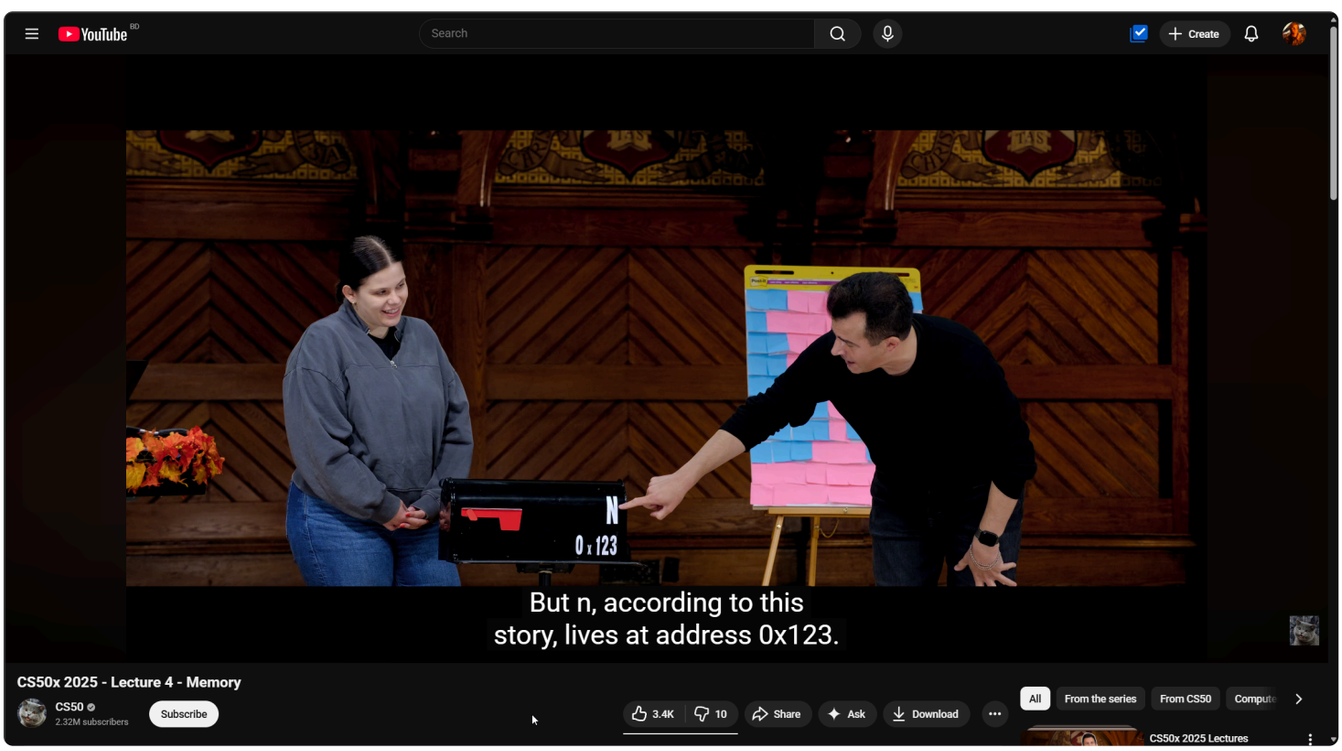
So, What is `p` ?
- *it's just a variable*
  - *whose value happens to be an address*

So, What is `n` ?
- *it simply happens to be*
  - *an integer at that address*



@18m47s 🔗 - Pointers
@20m35s 🔗 - "Address Of" Operator `&` ➡️ `int *p = &n;`
@22m13s 🔗 - Printing a Pointer ➡️ `0x7ffef2c2925c`
@25m00s 🔗 - `int* p;` or `int *p;` ?
@26m00s 🔗 - "De-Reference" Operator `*`
@27m45s 🔗 - *The designers of C, decages ago, made an "unfortunate decision"....*
@29m00s 🔗 - in terms of memory....
@30m16s 🔗 - FAQ





@33m00s - little smth to not lose today's memory

I. REYNEP. Highly recommend that you watch a few moments from here....

3.2.2. Pointers - `*` Operator - page2

```c
// prints an integer
#include <stdio.h>

int main(void)
{
  int  n = 50;

  printf("%i\n", n);    // we can change this line ➡️
}
```

1. *how we normally print* `int n;`

```c
// prints an integer
#include <stdio.h>

int main(void)
{
  int  n = 50;
  int *p = &n;
  printf("%i\n", *p);
}
```

2. `*p` ⟹ *use the location of* `n` *called* `p` *... and "go there", i.e. follow the breadcrumbs*

Two Operators

1. "Address Of" *Operator* `&`
2. "De-Reference" *Operator* `*`

The designers of C, decages ago, made, I would say the "unfortunate decision", to use the `*` star in 3 different ways in total:

1. *multiplication*
2. "De-Referencing" ➡️ `printf("%i\n", *p);`
3. "Creating" *a pointer* ➡️ `int *p;`

Admittedly Confusing, but

- `int *p = &n;` ⟹ *we are declaring a* "pointer" *&* *assigning it a memory-address*
- `printf("%i\n", *p);` ⟹ *we are* "De-Referencing" *that pointer.*

- *one sort of "creates" the pointer*
- *one "goes to" that address*

Copying DAVID's Lines from *@26m00s* 🔗 - "De-Reference" Operator `*`

> Well,
>
> 1. *I could print out* `n` *literally as I'm doing now.*
> 2. *Or, I could use the location of* `n` *called* `p` *.... and "go there" .... with the .... 2nd of these two operators*
>   - *The* Dereference Operator.
>   - *The* Dereference Operator - `*` *means "go to this address"*
>   - *I'm kind of jumping through hoops proverbially here, because this is the stupid, longer, more complicated way to print out the value of 50, but it just speaks to the relative power that I have by just 1) figuring out what the address is and then 2) going there.*

Copying DAVID's Lines from *@27m45s* 🔗 - *The designers of C, decages ago, made an "unfortunate decision"....*

> But the confusing part I do think about this for me and for most folks learning this is the following:
>
> - *The designers of C, decages ago, made, I would say the "unfortunate decision", to use the* `*` *star in 3 different ways in total:*
>
> *i. multiplication*
>
> *ii.* `printf("%i\n", *p);` ➡️ *here it literally means "go there"*
> - *i.e.* "DeReference" *the address*
> - *i.e. follow the breadcrumbs, and go there.*
>
> *iii.* `int *p;`
> - *But, when you declare the pointer*
> - *when you create the pointer*
>   - *you also use a* `*` *but*
>   - *you also use the* `type` .
>
> - *So, what's confusing at least to me, is, we are using the* `*` *in two different ways*
> - *both of which are related to addresses,*
> - *both of which are related to pointers*
>
> 1. *But, when you create a variable or declare a variable for the first time,*
> - `int *p;`
>   - *you specify not only* `*`
>   - *but the* `type` *of the pointer (just like any other variable declaration)*
> 2. *But, when you use the pointer subsequently,*
> - `printf("%i\n", *p);`
>   - *you do not mention the data type again.*
>   - *The compiler is smart enough to "remember"*
>   - *"okay, got it. you wanted a pointer to an integer." from the line*
>   - `int *p;`

3.2.3. String

```c
// Prints a string
#include <cs50.h>
#include <stdio.h>

int main(void)
{
  string s = "HI!";  // comes from <cs50.h>
  printf("%s\n", s);
}
```

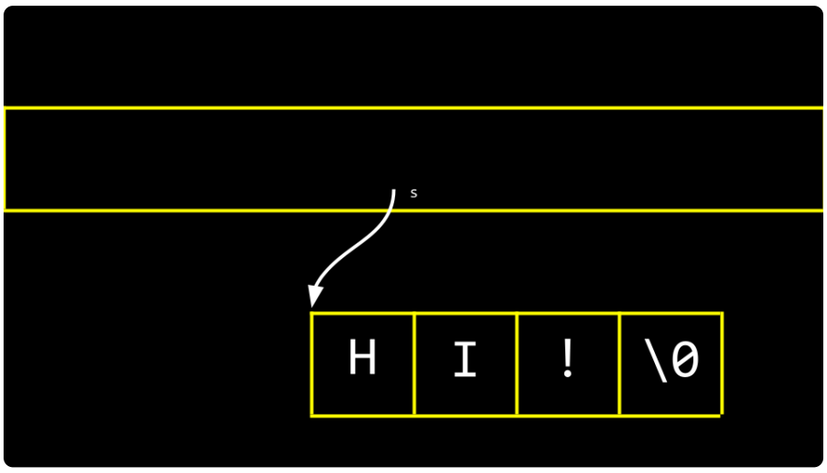
Now that we have a mental model for pointers, we can peel back a level of simplification that was offered earlier in this course.

Recall that a string is simply an "array" of characters.

- *For example,* `string s = "HI!"` *can be represented as follows:*



- *The idea of* "array" *will be introduced later...*



However, what is `s` really?

- *You can visualize the relationship of the variable* `s` *to the* `string` *as the following piece of memory:*

Notice

- *how a pointer called* `s` *tells the compiler where the first byte of the string exists in memory*

The idea of "array" will be introduced later...

- *but the key idea is*
  - `s[0]` $:= char_0$ `= 'H'`
  - `s[1]` $:= char_1$ `= 'I'`
  - `s[2]` $:= char_2$ `= '!'` *& so on*
  - *in CS, we love to start from* `0`

```c
// Prints each character of a string
#include <cs50.h>
#include <stdio.h>

int main(void)
{
    string s = "HI!";
    char k0 = s[0];     printf("%c\n", k0);     // outputs: H
    char k1 = s[1];     printf("%c\n", k1);     // outputs: I
    char k2 = s[2];     printf("%c\n", k2);     // outputs: !
    char k3 = s[3];     printf("%c\n", k3);     // outputs:
}
```

```c
// Prints a string's address
    // as well the addresses of its chars
#include <cs50.h>
#include <stdio.h>

int main(void)
{
    string s = "HI!";
    printf("%p\n", s);
    printf("%p\n", &s[0]);
    printf("%p\n", &s[1]);
    printf("%p\n", &s[2]);
    printf("%p\n", &s[3]);
}
```

This code

- *prints the memory locations of each character in the* `string s`.
- *The* `&` *symbol is used to show the address of each element of the string.*

- *When running this code, & looking @ the output, notice that*
  - *elements 0, 1, 2, and 3 of the string*
  - *a.k.a.* `s[0]`, `s[1]`, `s[2]`, `s[3]` *are next to one another in memory.*

3.2.4. `char *str`

```c
// Declares a string with CS50 Library
#include <cs50.h>
#include <stdio.h>

int main(void)
{
    string s = "HI!";
    printf("%s\n", s);
}
```

1. *using the* $cs50$ *training wheel* `string`

```c
// Declares a string without CS50 Library

#include <stdio.h>

int main(void)
{
    char *s = "HI!";
    printf("%s\n", s);
}
```

2. `cs50.h` *is removed.* `string` = `char *`
   - `char *c` *will represent the string that starts at the location of* `s`.
   - `char *c`
     - *is raw* C *code,*
     - *is without the scaffolding of the* $cs50$ *library.*
     - *effectively removes the training-wheels of the* `string` *data type offered by* `cs50.h`.

3.2.5. String - Pointer Arithmetic

```c
// Prints each character of a string
#include <stdio.h>

int main(void)
{
    char *s = "HI!";
    char k0 = s[0];     printf("%c\n", k0);     // outputs: H
    char k1 = s[1];     printf("%c\n", k1);     // outputs: I
    char k2 = s[2];     printf("%c\n", k2);     // outputs: !
    char k3 = s[3];     printf("%c\n", k3);     // outputs:
}
```

```c
// Prints a string's chars
#include <stdio.h>

int main(void)
{
    char *s = "HI!";
    printf("%c\n", s[0]);   // same effect
    printf("%c\n", s[1]);
    printf("%c\n", s[2]);
    printf("%c\n", s[3]);
}
```

```c
// Prints a string's chars
#include <stdio.h>

int main(void)
{
    char *s = "HI!";
    printf("%c\n", s[0]);
    printf("%c\n", s[1]);
    printf("%c\n", s[2]);
}
```

1. *printing each* `char` @ `s`

```c
// Prints a string's chars via pointer arithmetic
#include <stdio.h>

int main(void)
{
    char *s = "HI!";
    printf("%c\n", *s);
    printf("%c\n", *(s + 1));       // Pointer Arithmetic
    printf("%c\n", *(s + 2));       // Eventually having the same effect
}
```

2. *Notice that*
   i. *the* 1st *character at the location of* `s` *is printed.*
   ii. *Then, the character at the location* `s + 1` *is printed, and so on.*
   iii. *effectly having the same effect as the code on the left*

3.2.6. Loops

```c
#include <stdio.h>

int main(void)
{
    printf("meow\n");
    printf("meow\n");
    printf("meow\n");
}
```



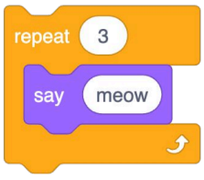
@1h23m26s - Loops

```c
#include <stdio.h>

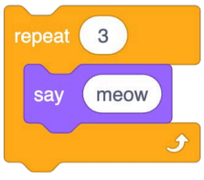
int main(void)          // Code with Same Effect
{
    int i = 1;
    while (i <= 3)
    {
        printf("meow\n");
        i = i+1;
    }
}
```

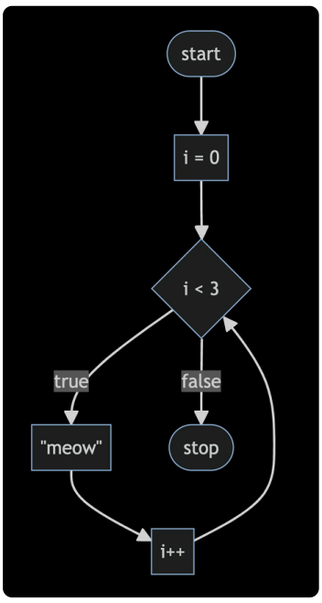
- `i = i+1` ≔ *Every time* `1` *is added to* `i`
  - *Once* `i` *is greater than 3, it will stop the loop.*

```c
int i = 1;
while (i <= 3)
{
    printf("meow\n");
    i = i+1;
}
```




```c
int i = 0;
while (i < 3)
{
    printf("meow\n");
    i++;
}
```

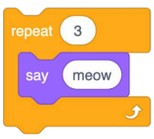





```c
int i = 0;
while (i < 3)
{
    printf("meow\n");
    i++;
}
```

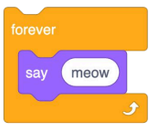
```c
for (int i = 0; i < 3; i++)
{
    printf("meow\n");
}
```

3.2.6. Loops - Summary



1. repeats 3 times

```c
for (int i = 0; i < 3; i++)
{
    printf("meow\n");
}
```



1. repeats forever

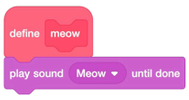
```c
while (true)
{
    printf("meow\n");
}
```

```c
int n = 3;      // also we could have a variable like this

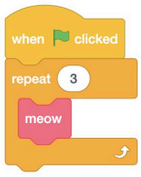
for (int i = 0; i < n; i++)     // that plug that in here
{
    printf("meow\n");
}
```

3.2.7. Functions

1. *These two left-images together*
   - *"no input"*
   - *"no output"*

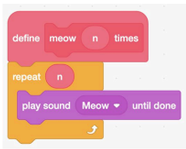



```c
void meow(void)
{
    printf("meow\n");
}
```

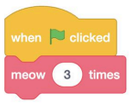



```c
int main(void)
{
    for (int i = 0; i < 3; i++)
    {
        meow();
    }
}
```

1. *These two right-images together*
   - `int n` *input*
   - *"no output"*




```c
void meow(int n)
{
    for (int i = 0; i < n; i++)
    {
        printf("meow\n");
    }
}
```




```c
int main(void)
{
    meow(3);
}
```

```c
#include <stdio.h>

// Meow once
void meow(void)
{
    printf("meow\n");
}



int main(void)
{
    for (int i = 0; i < 3; i++)
    {
        meow();
    }
}
```

```c
#include <stdio.h>

// Meow some number of times
void meow(int n)
{
    for (int i = 0; i < n; i++)
    {
        printf("meow\n");
    }
}



int main(void)
{
    meow(3);
}
```

Printing
- *is* **not** *an "output" from the perspective of "functions".*
- *is an "output" from the perspective of user & the terminal*

"Functions"
- *inputs are called "parameters" -- can be multiple inputs*
- *output is called "return value" -- must return only 1 thing.*

- *Why do functions need inputs?*

```c
int main(void) {
    int k = 10;
}

int func(void) {
    printf("%d", k); printf("\n");
}
```

  - *Because, this code won't compile....*
  - `k` *is not accessible outside* `main`



@1h34m37s
Functions

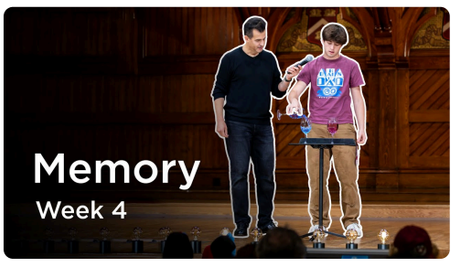
```c
// Meow some number of times
void meow(int n)
{
    for (int i = 0; i < n; i++)
    {
        printf("meow\n");
    }
}
```

```c
// Get number of meows
int get_positive_int(void)
{
    int n = -1;
    while (true)
    {
        n = get_int("Number: ");
        if (n > 1) {
            break;      // break the look
        }
    }
    return n;
}
```

```c
#include <cs50.h>
#include <stdio.h>

int main(void) {
    int n = get_positive_int();
    meow(n);
}
```

3.2.8. Swapping



In the real world

- *a common need in programming is to swap two values.*
- *Naturally, it's hard to swap two variables without a temporary holding space.*
- *In practice, you can type code* `swap.c` *and write code as follows to see this in action:*

@1h39m00s
swapping

The Problem

```c
// Fails to swap two integers
#include <stdio.h>

void swap(int a, int b)
{
    int tmp = a;
    a = b;
    b = tmp;
}

int main(void)
{
    int x = 1;
    int y = 2;

    printf("x is %i, y is %i\n", x, y);
    swap(x, y);
    printf("x is %i, y is %i\n", x, y);
}
```

```
output
    x is 1, y is 2
    x is 1, y is 2
```

Notice that

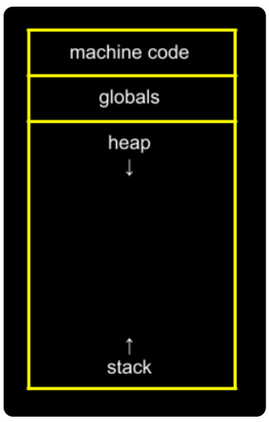
- *while this code runs, it does not work.*
- *The values, even after being sent to the* swap() *function, do not swap.* Why?

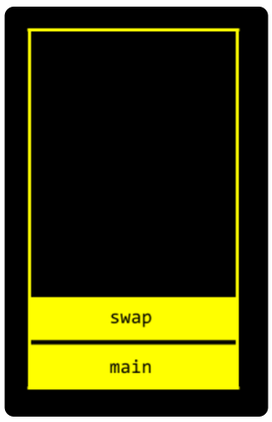
When you pass values to a function,

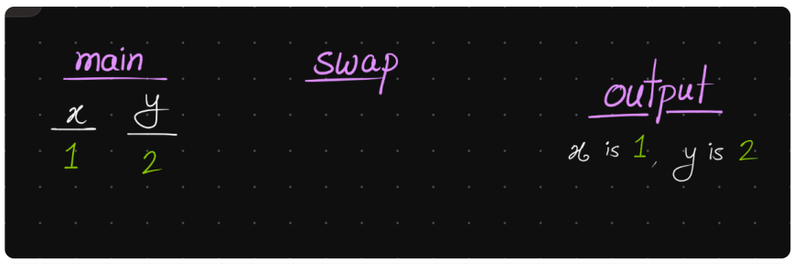
- *you are only providing copies.*
- *In our code above,* `x` *and* `y` *are being passed by value.*
  - *whenever, this is done, values(* `x` & `y` *) are getting copied into* `a` & `b`

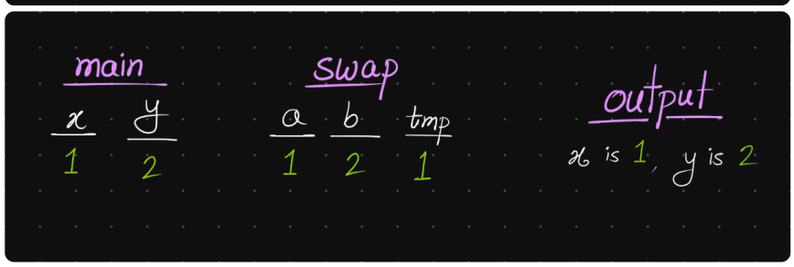
Two Different Functions

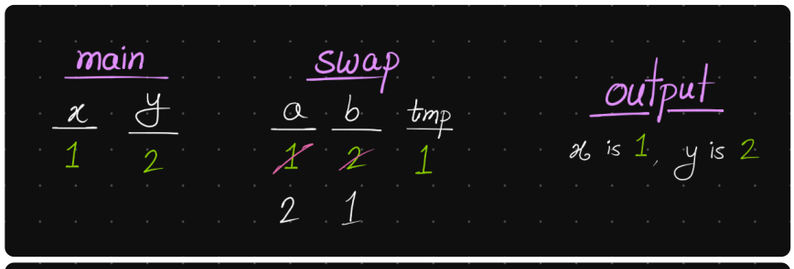
- `main` *and* `swap` *, in terms of memory,*
  - *have two separate frames or areas of memory.*











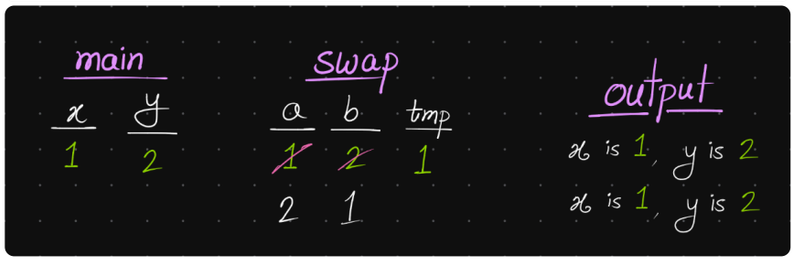


Image 1

- *executed the first 3 lines of* `main()`

Image 2

- *line 4 of* `main()` ⟹ `swap(x, y);`
- `swap(x, y)` *has been called*
  - *so we must go inside this function*
  - *and execute code line by line*
  - *we executed line 1 inside* `swap()`

Image 3

- *line 2 inside* `swap` ➡️ *update* `a`
- *line 3 inside* `swap` ➡️ *update* `b`
- *no more code inside swap, the end*
  - *so we* `return`

Image 4

- *Even tho,* `a` & `b` *might have been swapped*
- `x` , `y` *was not changed* 🤷‍♀️

3.2.8. Swapping

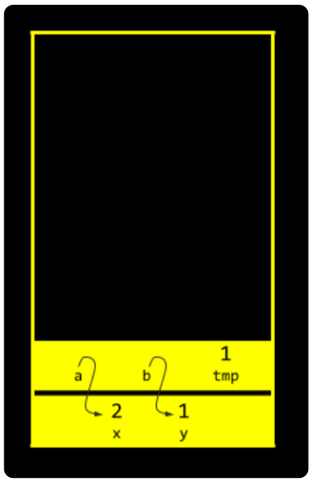
The Solution

```c
// Swaps two integers using pointers
#include <stdio.h>

void swap(int *a, int *b)
{
    int tmp = *a;
    *a = *b;
    *b = tmp;
}

int main(void)
{
    int x = 1;
    int y = 2;

    printf("x is %i, y is %i\n", x, y);
    swap(&x, &y);
    printf("x is %i, y is %i\n", x, y);
}
```



Don't Get Confused!

Remember,
The designers of *C*, decages ago, made, an "unfortunate decision":

- *Star* `*` *is used in 3 different ways in total:*
  i. **multiplication**
  ii. **De-Referencing** ➡ `printf("%i\n", *p);`
  iii. **Creating a pointer** ➡ `int *p;`

```c
void swap(int *a, int *b)
```

- `int *a` ➡ "Creating" *a pointer*

```c
int tmp = *a;
```

- `*a` ➡ "De-Referencing" *a pointer*

```c
*a = *b;
```

- `*a` ➡ *"follow the breadcrumbs", go to that address,* **save** *value in there*
- `*b` ➡ *"follow the breadcrumbs", go to that address,* **grab** *value from there*

This is called,

- *"passing value by reference"*
  - *That is*
    - *instead of copying values(* `x` & `y` *)*
    - *the addresses of* `x` *and* `y` *are provided to the function*
  - *And*
    - *if we know the address*
    - *we can just "go there" & poke around*

3.2.9. the `scanf()` function

```c
// Gets an int from user using scanf
#include <stdio.h>

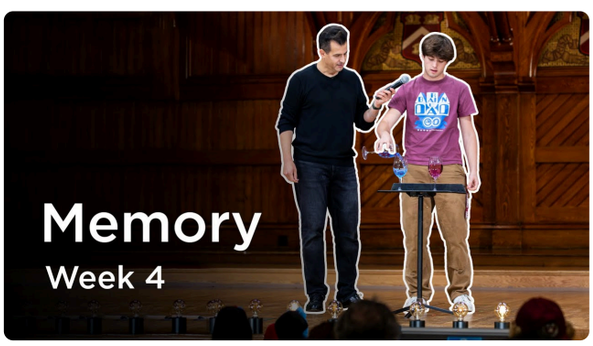
int main(void)
{
    printf("Enter a Number (n): ");
        int n;
        scanf("%i", &n);
            // %i ==> scanf() will expect an integer
            // &n ==> scanf() will store the integer @ the address "&n"
            //  n ==> is not a address, it's a variable
            // &n ==> is the actual address

    printf("n: %i\n", n);
}
```

Now, if you understood, why the swapping function needs address of `x` and `y`, then you could guess, why `scanf()` also needs address.

```c
int m, n;
float x, y;
scanf("%d %d %f %f", &m, &n, &x, &y);
```

```c
1 -20 .3 -4.0e3
  // Say, the user writes this line and hits enter
  // Then
        m = 1       x = 0.3
        n = -20     y = -4000.0
```



@2h01m24s
scanf()

```c
#include <stdio.h>

int main(void) {
    char *s;    // buggy
    printf("s: ");
    scanf("%s", s);
    printf("s: %s\n", s);
}
```

1. `s` *points to a different address everytime you run it*

```c
#include <stdio.h>

int main(void) {
    char s[4];  // fixed (somewhat)
    printf("s: ");
    scanf("%s", s);
    printf("s: %s\n", s);
}
```

2. `s` *is still a pointer.* `s` *now points to an* `char` *array of size* `4`
   - *we can input* `cat` *and the program functions.*
   - *However, an input larger than this could create an error.*
     - *so,* `char s[1024];` *or more could be done*

3.2.10. `scanf()` - ivans lines

Recall that
- `printf()` *prints output according to a really specific* **format string**
- `printf()` *prints output according to a specified format*

format string consists of:
- **ordinary characters**
- **conversion specifications**

Just the Same way
- `scanf()` *reads input according to a particular format too.*
  - *so* `scanf()` *takes in a* **format string** *too.*
  - *The conversions allowed with* `scanf()` *are essentially the same as those used with* `printf()`.
  - *A* `scanf()` **format string** *may contain both ordinary characters and conversion specifications.*

```
int m, n;
float x, y;
scanf("%d %d %f %f", &m, &n, &x, &y);
```

```
1 -20 .3 -4.0e3
```

- *(then up next,* **ivan** *showed this code & didn't even bother to write any text)*
- **ivan** *also called this input above, "output"*

In many cases
- *a* `scanf()` **format string** *will contain only these conversion specifications, as in the following example:*

Suppose that the user enters the following input line:
and then presses enter.
- `scanf()` *will read the line*
- *converting its characters to the numbers they represent,*
- *and then*
  - **assign** `1`, `-20`, `0.3`, *and* `-4000.0`
  - *to* `m`, `n`, `x`, *and* `y`, *respectively.*

```
scanf("%d %d %f %f", &m, &n, &x, &y);
scanf("%d%d%f%f", &m, &n, &x, &y);
```

Note that
- *both line here, would have had the same effect.*
  - *"Tightly packed"* **format strings** *like* `"%d%d%f%f"` *are common in* `scanf()` *calls.*

3.2.11. `scanf()` traps (unedited)

`scanf()` & `printf()` contains several traps for the unwary. When using them, the programmer must check that:

1. *the number of* **conversion specifications** *matches the number of* **input variables**
2. **each conversion** *is appropriate for the corresponding variable*
   - *the compiler isn't required to check for a possible mismatch.*
3. *Another trap for* `scanf()` *involves the* `&` *symbol*
   - `&` *normally precedes each variable in a* `scanf()` *call.*
   - *The* `&` *is usually (but not always) required, and it's the programmer's responsibility to remember to use it.*

3.2.12. How `scanf()` works 🟩 (unedited)

Uhem,

- `scanf()` *can actually do much more than I've indicated so far.*
- `scanf()` *is essentially a* **"pattern-matching"** *function*
  - *that tries to match up groups of input characters with conversion specifications in a format-string*
- `scanf()` & `printf()` *is controlled by the format string.*

When `scanf()` is called,

- `scanf()` *begins processing the information in the string, starting at the left.*
- 🟩 *For each* **conversion specification** *in the format string*
  - `scanf()` *tries to locate an item of the appropriate type in the input data,*
    - *skipping blank space if necessary.*
  - `scanf()` *then reads the item*
    - *stopping when it encounters a character that can't possibly belong to the item.*
  - *If the item was read successfully,*
    - `scanf()` *continues processing the rest of the format string.*
  - *If any item is not read successfully,*
    - `scanf()` *returns immediately*
      - *without looking at the rest of the format string (or the remaining input data).*
  - *As* `scanf()` *searches for the beginning of a number, it* **ignores** white-space characters

---

- *Remember,* **white-space characters**
  - ` ` - ASCII 32 - **space**
  - `\n` - ASCII 10 - **newline** - *Moves cursor to the beginning of the next line*
  - `\t` - ASCII 9 - **horizontal tab**
  - `\v` - ASCII 11 - **vertical tab**
  - `\f` - ASCII 12 - **form feed** - *Advances the output to the next page or form*
  - `\r` - ASCII 13 - **carriage return** - *Moves cursor to the beginning of the current line*

- *As a result, numbers can be put on a single line or spread out over several lines.*
  Consider the following call of `scanf()`*:*

```
scanf("%d%d%f%f", &i, &j, &x, &y);
```

Suppose that the user enters three lines of input:

```
1
-20   .3
-4.0e3
```

`scanf()` *sees one continuous stream of characters:*

```
··1¤-20···.3¤···-4.0e3¤
```

- *Author's using* `·` *to represent space and* `¤` *to represent the new-line.*

```
··1¤-20···.3¤···-4.0e3¤
ssrsrrrssrrsssrrrrrr
```

- `s` *under a character indicates that it was skipped*
- `r` *indicates it was read as part of an input item*

- 🎴 `scanf()` **"peeks"** *at the final* **new-line** *character without actually reading it.*
- 🎴 *This* **new-line** *will be the first character read by the next call of* `scanf()`.

3.2.13. How `scanf()` works - II - 🟩 (unedited)

What rules does `scanf()` follow to recognize an `int` or `float`?

- 🟩 *When asked to read an* `int`
  - `scanf()` *first searches for*
    - *a digit, or*
    - *a plus sign, or*
    - *a minus sign;*
  - *it then*
    - *keeps on reading digits until it reaches a non-digit.*

- 🟩 *When asked to read a* `float`
  - `scanf()` *first looks for*
    - *a digit, or*
    - *a plus sign, or*
    - *a minus sign;*
  - *followed by*
    - *more digits, or,*
    - *decimal point + more digits*
  - *followed by*
    - *an exponent (optional).*
    - *a.k.a.* `e+00` *or* `E+00`
      - *An exponent consists of the letter* `e` *(or* `E`*), an optional sign, and one or more digits.*
  - *The* `%e`, `%f`, *and* `%g` *conversions are interchangeable when used with* `scanf()`*;*
    - *all three follow the same rules for recognizing a* `float`

3.2.14. How `scanf()` works - III : Matching Conv. Specs. 🟦 TextBook

🎴 When `scanf()` encounters a character that can't be part of the current item,

- *the character is* **"put back"** *to be read again*
  - *during the scanning of the* **next input item** *(next* `%` *stuff) or*
  - *during the* **next call** *of* `scanf()`.

```
INPUT: 1-20.3-4.0e3¤

scanf("%d%d%f%f", &m, &n, &x, &y);
    // Let's use the same call of scanf as before
```

Consider the following (admittedly pathological) arrangement of our four numbers:

Here's how `scanf()` would process the new input:

1. `%d`
   - *The first nonblank input character is* `1`*;*
     - *since integers can begin with* `1`*, scanf then reads the next character,* `-`.
   - *Recognizing that* `-` *can't appear inside an integer,*
     - `scanf()` *stores* `1` *into* `m` *and*
     - *puts the* `-` *character back.*
       - *for it shall be processed by the next call of* `scanf()` *or the next* `%` *item.*
2. `%d`
   - `scanf()` *then reads the characters* `-`, `2`, `0`, *and* `.` *(period).*
     - *The 4th character it read was* `.` *Since an* `int` *can't contain a decimal point,*
       - *scanf stores* `-20` *into* `m` *and*
       - *puts the* `.` *character back.*
3. `%f`
   - `scanf()` *reads the characters* `.`, `3`, *and* `-`.
     - *Since a* `float` *can't contain a minus sign after a digit,*
       - `scanf()` *stores* `0.3` *into* `x` *and*
       - *puts the* `-` *character back.*
4. `%f` *- Last one,*
   - `scanf()` *reads the characters* `-`, `4`, `.`, `0`, `e`, `3`, *and* `¤` *(new-line).*
     - *Since a* `float` *can't contain a* **new-line** *character,*
       - `scanf()` *stores* -4.0×10^3 *into* `y` *and*
       - *puts the* **new-line** *character back.*
   - 🎴 *NOTE: this* `¤` *(new-line)* **will be read by the next** `scanf()`

In this example,

- `scanf()` *was able to match every* **conversion specification** *in the format string with an input item.*
- 🎴 *Since the* **new-line** *character wasn't read,*
  - **it will be left for the next call of** `scanf()`.

3.2.15. How `scanf()` works - IV : Ordinary Characters 🟩

Consider

```
scanf("%d SMTH %d", &x, &y);
scanf("%d/%d", &x, &y);
```

🎴 🟩 When `scanf()` encounters a character that can't be part of the current item,

- *the character is* **"put back"** *to be read again*
  - *during the scanning of the* **next input item** *(next* `%` *stuff) or*
  - *during the* **next call** *of* `scanf()`.

> The concept of pattern-matching can be taken one step further by writing format strings that contain ordinary characters in addition to conversion specifications. The action that scanf takes when it processes an ordinary character in a format string depends on whether or not it's a white-space character.

3 Type of Characters in **Format-String** or The Input **:**

1. 🟩 **Matching Characters /w Conv. Specs.**
   - *Well, we know what* `scanf()` *does to these ones* 🤷
   - *Everything else other than* **matching chars** ➡️ **Ordinary Characters**
2. 🟩 **White-space characters**

- *Remember,* **white-space characters**
  - ` ` - ASCII 32 - **space**
  - `\n` - ASCII 10 - **newline** - *Moves cursor to the beginning of the next line*
  - `\t` - ASCII 9 - **horizontal tab**
  - `\v` - ASCII 11 - **vertical tab**
  - `\f` - ASCII 12 - **form feed** - *Advances the output to the next page or form*
  - `\r` - ASCII 13 - **carriage return** - *Moves cursor to the beginning of the current line*

- *When* `scanf()` *encounters one or more* **white-space characters** *in a* **format string**
  - `scanf()` *reads all* **white-space characters** *from the input until it reaches a* **non-white-space character.**
- *So, if* `scanf()` *finds*
  - `1` *Space or*
  - `1` *single* `\t` *or*
  - `1` *single* `\n`
    - *then, it will* **keep discarding all** *the* **white-space characters** *together*
    - *untill it finds a* **non-white-space**
- *This* **non-white-space** *is now gonna be compared with the next character in our* **format-string**
- 🎴 *Now,* `1` **white-space char.** *in the* **format-string** *will match any number of* **white-space char.** *in the input.*
- *But, putting a* **white-space char.** *in a* **format-string** doesn't force **the input** *to contain* **white-space char.**

3. 🟩 **Other characters** ➡️ `SMTH`
   - *When* `scanf()` *encounters a* **non-white-space** *character in a* **format string**
     - `scanf()` *compares it with the next input character*
       - *If they match* ➡️ *The* `scanf()` *discards the input character and continues processing* **format string**
       - *If they don't match* ➡️ `scanf()` **puts** *the offending character* **back into the input**, *then* **aborts**

- 🟩 *Examples:*

```
scanf("%d/%d", &x, &y);

input: ·5/·96
    x=5, y=96
    Everything Works Out!
```

```
scanf("%d/%d", &x, &y);

input: ·5·/·96
    scanf() fails
    the / in the "format-string" doesn't match the space in the input
```

  - *To allow spaces after the first number,*
    - *we should use the* **format-string** `"%d /%d"` *instead.*

3.2.16. Confusing `printf()` with `scanf()` 🟩 (unedited)

- *Don't use ordinary characters in* `scanf()` *unless it is necessary*

```
scanf("%d, %d", &i, &j);    // Not error, but be careful
```

first, `scanf()` *will look for an* `int` *in the input* ➡️ *stores in* `i`

then, `scanf()` *takes the next character in the* **format-string**

  - `,`
    - *This is not a* **1. Conv. Specs.**
    - *This is not a* **2. White-Space Character**
    - *This is a* **3. Ordinary Other Character**

  - *Take a look back @ the* **3. Other Character** *Handling rules from last slide* 😊

  - *If the next input character is a* ` ` *space, not a* `,` *comma*
    - *then* `scanf()` *realizes that they don't match*
    - *and as per the rules described in the last slide,*
      - `scanf()` *will terminate without reading a value for `j.*

- *Don't put a* **new-line character** *at the end of a* `scanf()` **format-string**

```
scanf("%d\n", &i);
scanf("%d \n", &i);
scanf("%d \n ", &i);
```

- *If the* **format-string** *is* `"%d\n"`,
  `scanf()` *will*
  - `%d`
    - *skip* **2. White-space characters**
    - *read an integer, by finding* **1. Matching characters**
      - *stop as soon as it hits a* **non-digit** ➡️ *let's call that* **3. Other characters**

  - *So this* **3. Other character** *is now gonna be compared with the next character in the* **format-string**, *which apparently is* `\n`

  - `\n`
    - *is itself a* **2. White-space characters**
    - *This* **trailing whitespace** *in our* **format-string** *tells* `scanf()` *to keep reading and discarding any and all* **2. whitespace**

  - *The Stopping Condition:*
    - `scanf()` *only finishes that trailing* `\n` *directive*
      when `scanf()` *encounters something that is not a* **2. whitespace** *(like a letter or another number)*

    - *So, if we enter* `50\n`
      - `scanf()` *won't end / won't* `return` *just yet*
      - *thus --> cause interactive programs to hang*

- *If the* **format-string** *is* `"%d \n"` ➡️ *Same would happen....*
  - *Because* ` ` *(space) &* `\n` *are both* **2. White-space characters**
- *If the* **format-string** *is* `"%d \n "` ➡️ *Same would happen....*

- *Don't use ampersand in* `printf()`

```
printf("%d %d\n", &i,&j);   // Wrong
```

3.2.17. Program : `Adding Fractions`

```
Enter first fraction: 5/6
Enter second fraction: 3/4
The sum is 38/24
```

- *Consider a* `C` *program that prompts the user to enter two fractions and then displays their sum*
- *Fractions are customarily denoted as: numerator/denominator*

Solution

```c
#include <stdio.h>

int main(void)
{
    int n1, d1;
        printf("Enter first fraction: ");
        scanf("%d / %d");

    int n2, d2;
        printf("Enter second fraction: ");
        scanf("%d / %d");

    int lcm_d = find_lcm(d2, d2);
    int d = lcm_d;
    int n = (n1*(d/d1)) + (n2*(d/d2))

    printf("The sum is %d/%d", n, d);
}
```

```c
// Function to find GCD using Euclid's algorithm (Recursive)
int find_gcd(int a, int b) {
    if (b == 0)
        return a;
    return find_gcd(b, a % b);
}

// Function to find LCM using the GCD result
int find_lcm(int a, int b) {
    return (a * b) / find_gcd(a, b);
}
```

$$LCM(a,b) = \frac{|a \times b|}{\text{GCD}(a,b)}$$

REYNEP's yap yap

So, remember `get_int()`, `get_float()`, `get_string()` from week 1 (cs50)?
These are sorta' like some really nice training wheels on a bicycle 🤷‍♀️ that cs50 gives us.

Now, even cs50 doesn't show `scanf()` untill before the middle of the course, cz `scanf()` is very sophisticated, one might even say that it is goddamn scary.

cs50 introduces `scanf()` @ 2h01m of week 4, that's like the 12th hour of this course 🤷‍♀️.
and the course is 24hours altogether.

`scanf()` also requires for the user to have knowledge about pointers, which is introduced in week 4.

The End